Graph search

BFS, shortest paths and A*: queues and heaps become layers

The shape of a search algorithm

Chapters 1 to 3 gave us the toolbox: masks, `select`, one-hot reads and writes, `argmin`, and loops unrolled into a fixed number of identical layers. This chapter puts them to work on the three classic graph-search algorithms. All three have the same skeleton:

> **Key idea**
>
> **State** is a handful of vectors with one entry per node ("is it visited?", "what is its distance so far?"). One **layer** is one round of the algorithm: **select** some nodes, **expand** them (along edges), **update** the vectors. Run the same layer a fixed number of times. A **halt flag** makes the layers after the finish a no-op.

What changes between BFS, Dijkstra / Bellman-Ford and A* is only the **select** step:

Algorithm	select	why it becomes data flow
BFS	every node of the current layer, at once	a layer of nodes needs no queue: all of it is the frontier vector
Bellman-Ford	every edge, every round	nothing is selected: relax everything, `min` keeps the best
A*	the single open node with the lowest f	it stays sequential: one `argmin` one-hot per layer, N layers

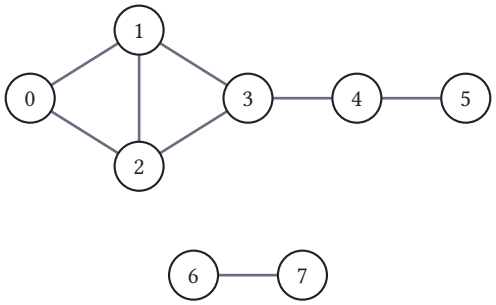
Each example below is the same three stages as before. There are two ways to hold a graph. **In the weights:** the adjacency matrix is a constant matrix, built at import time (BFS part 1; the grid's neighbours in A*). **In the input:** the edges (BFS part 2), the edge weights (shortest paths) or the walls (A*) are runtime numbers, and the network has to combine them with the state using ANDs, sums and selects instead of a constant matrix.

14 BFS: a queue becomes a wave

basic_python/04_graph_search/14_bfs.py

The problem

Given an undirected graph and a start node, find the number of edges on a shortest path from the start to every node, and -1 for nodes the start cannot reach. Breadth-first search does this by visiting nodes in order of distance, with a FIFO queue.



The example graph has 8 nodes. The component $\{6, 7\}$ is cut off from the rest, so starting from node 0 those two stay at -1. Everything below uses this graph; start node 0 unless stated otherwise.

A and B side by side

A · control flow

```python
def control_flow(adj, start):
    dist = [-1] * len(adj)
    dist[start] = 0
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v in range(len(adj)):
            if adj[u][v] and \
                    dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

B · branchless arithmetic

```python
def branchless(start):  # one-hot
    visited = list(start)
    frontier = list(start)
    dist = [0] * V
    for _ in range(V - 1):
        reached = [
            sum(ADJ[v][u] * frontier[u]
                for u in range(V))
            for v in range(V)]
        new = [step(reached[v]
                    - K * visited[v])
               for v in range(V)]
        dist = [dist[v] + 1 - visited[v]
                for v in range(V)]
        visited = [visited[v] + new[v]
                   for v in range(V)]
        frontier = new
    return [dist[v] + V * visited[v] - V
            for v in range(V)]
```

Nothing in B depends on the **values**: there is no `while queue`, no `if`, no `dist[v] == -1`. The only things that look at data are `relu` (inside `step`) and arithmetic.

Why the queue disappears

A queue is a way of processing nodes one at a time while keeping them in distance order. But look at what is in the queue at any moment: nodes of at most two **consecutive** distances. BFS is secretly **level-synchronous**: first every node at distance 1 gets processed, then every node at distance 2, and so on. A level never depends on the order inside the previous level, only on the **set** of nodes in it. So instead of a queue, keep that set as a 0/1 vector, the **frontier**, and process all of it in one layer.

> **Key idea**
>
> Queue (ordered list, one node at a time) → **frontier vector** (a set, a whole level at a time). The order inside a level never mattered, so nothing is lost.

The state is three vectors, one entry per node:

vector	values	meaning
`visited`	0/1	nodes reached so far (the "dist[v] != −1" test, as a vector)
`frontier`	0/1	nodes reached by the **last** wave, i.e. the nodes to expand now
`dist`	integers	distance accumulator (explained below)

One layer = one wave

$$\begin{aligned}\text{reached} &= A\,\text{frontier}\\ \text{new} &= \text{step}(\text{reached} - K\cdot\text{visited})\\ \text{visited} &\leftarrow \text{visited} + \text{new}\\ \text{frontier} &\leftarrow \text{new}\end{aligned}$$

The adjacency matrix is the weight matrix. A is the 0/1 matrix with $A_{vu} = 1$ when u and v are joined by an edge. For a frontier vector f, $(Af)_v = \sum_u A_{vu} f_u$ counts how many frontier nodes neighbour v. This single matrix-vector product is "for each node in the frontier, look at all its neighbours", done for all frontier nodes at once. For our graph:

$$A = \begin{bmatrix} 0&1&1&0&0&0&0&0\\ 1&0&1&1&0&0&0&0\\ 1&1&0&1&0&0&0&0\\ 0&1&1&0&1&0&0&0\\ 0&0&0&1&0&1&0&0\\ 0&0&0&0&1&0&0&0\\ 0&0&0&0&0&0&0&1\\ 0&0&0&0&0&0&1&0 \end{bmatrix}$$

Row 3 reads "node 3 touches nodes 1, 2 and 4". The largest row sum is 3 (nodes 1, 2 and 3): the **maximum degree**.

Filtering the already visited: why K. `reached[v] > 0` only says "some frontier node touches v". We want "… and v was never visited". The data-flow way to say "and not" is to subtract a big number when the bad thing is true:

$$p_v = \text{reached}_v - K\cdot\text{visited}_v \qquad \text{new}_v = \text{step}(p_v)$$

v visited?	reached	p_v	$\text{new}_v = \text{step}(p_v)$
no	0	0	0: nobody touches it
no	≥ 1	≥ 1	1: newly discovered
yes	$0..K$	≤ 0	0: never again, as long as $K \geq$ reached

So K must be at least the largest value reached_v can take. That is the number of frontier neighbours of v, at most the maximum degree. Here $K = 3$. Too small a K is a real bug, and the trace below shows it.

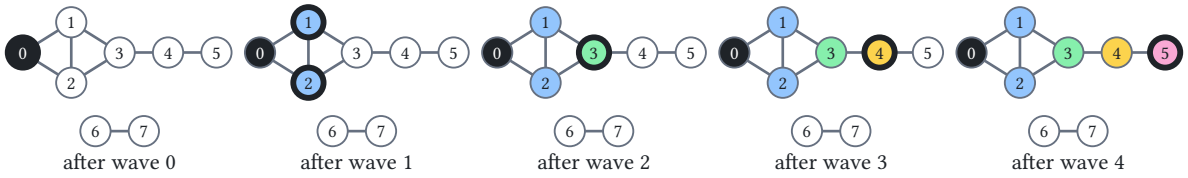
Distances without a layer counter. The textbook writes `dist[v] = dist[u] + 1`; the obvious data-flow version adds `t * new` in layer t, which would need a different constant per layer. We want $V - 1$ **identical** layers, so use this instead: every layer, every node that is **not yet visited** gets older by one.

$$\text{dist} \leftarrow \text{dist} + 1 - \text{visited} \quad (\text{visited before this wave})$$

A node discovered by wave d was unvisited at the start of waves $1, \ldots, d$, so it ages d times and ends at d. The start node is visited from the beginning and stays at 0. A node never reached ages in all $V - 1$ waves and ends at $V - 1$; a final affine read-out $\text{dist} + V\cdot\text{visited} - V$ turns that into -1 (reachable nodes have visited $= 1$ and are left alone).

How many layers? A shortest path has at most $V - 1$ edges, so $V - 1$ waves always suffice. After the last real wave the frontier is all zeros, so the remaining layers change nothing: **halting is free** in BFS, the wave simply dies out.

Worked trace (start node 0)



Colour = the wave that discovered the node. The thick outline is the current frontier.

Numbers, in the order the layers compute them (node numbers 0 to 7 left to right inside each vector):

wave	frontier f	reached $= Af$	new	visited after
1	1 0 0 0 0 0 0 0	0 1 1 0 0 0 0 0	0 1 1 0 0 0 0 0	1 1 1 0 0 0 0 0
2	0 1 1 0 0 0 0 0	2 1 1 2 0 0 0 0	0 0 0 1 0 0 0 0	1 1 1 1 0 0 0 0
3	0 0 0 1 0 0 0 0	0 1 1 0 1 0 0 0	0 0 0 0 1 0 0 0	1 1 1 1 1 0 0 0
4	0 0 0 0 1 0 0 0	0 0 0 1 0 1 0 0	0 0 0 0 0 1 0 0	1 1 1 1 1 1 0 0
5	0 0 0 0 0 1 0 0	0 0 0 0 1 0 0 0	0 0 0 0 0 0 0 0	1 1 1 1 1 1 0 0

Wave 2 is the interesting one. Frontier $\{1, 2\}$ touch each other and node 0 **and** node 3. Af is $[2, 1, 1, 2, 0, \ldots]$: node 0 is hit twice and node 3 is hit twice. With $K = 3$ and visited $= [1, 1, 1, 0, \ldots]$ we get $p = [-1, -2, -2, 2, 0, \ldots]$, so only node 3 is new. With $K = 1$ we would get $p_0 = 2 - 1 = 1$ and node 0 would be "discovered" again: its distance would be corrupted and `visited[0]` would become 2. A visited node that is hit by several frontier nodes needs $K \geq$ that count.

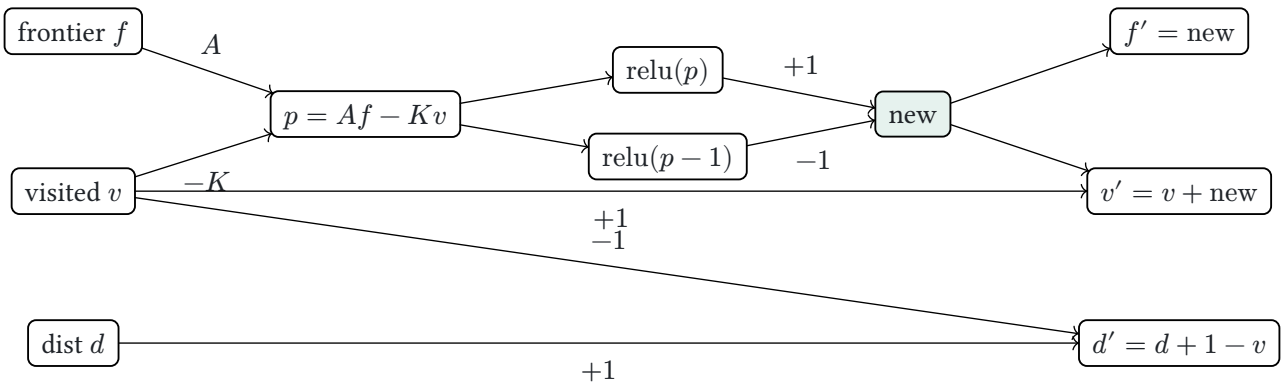
The `dist` accumulator along the way (all 8 nodes): $[0, 0, 0, 0, 0, 0, 0, 0] \to [0, 1, 1, 1, 1, 1, 1, 1] \to [0, 1, 1, 2, 2, 2, 2, 2] \to [0, 1, 1, 2, 3, 3, 3, 3] \to [0, 1, 1, 2, 3, 4, 4, 4]$, and then $[0, 1, 1, 2, 3, 4, 5, 5]$, $[0, 1, 1, 2, 3, 4, 6, 6]$, $[0, 1, 1, 2, 3, 4, 7, 7]$ after the idle waves 5 to 7. The read-out dist $+ 8 \cdot$ visited $- 8$ gives $[0, 1, 1, 2, 3, 4, -1, -1]$ since nodes 6 and 7 have visited $= 0$ and $7 + 0 - 8 = -1$.

Stage C: the weights

State $x = [$ visited $\mid$ frontier $\mid$ dist $]$ has $3V = 24$ numbers, all ≥ 0. That matters: for $x \geq 0$, $\text{relu}(x) = x$, so a ReLU neuron can **copy** a number through a layer. One layer is $x' = W_2\,\text{relu}(W_1 x + b_1) + b_2$ with hidden vector $h = [\text{relu}(p) \mid \text{relu}(p-1) \mid$ visited $\mid$ frontier $\mid$ dist$]$:

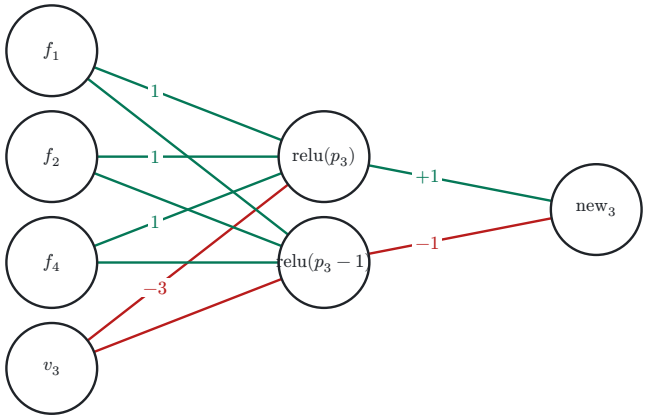
$$W_1 = \begin{pmatrix} -KI & A & 0 \\ -KI & A & 0 \\ I & 0 & 0 \\ 0 & I & 0 \\ 0 & 0 & I \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad W_2 = \begin{pmatrix} I & -I & I & 0 & 0 \\ I & -I & 0 & 0 & 0 \\ 0 & 0 & -I & 0 & I \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Each block is 8×8 (I = identity, 0 = zero block; b_1, b_2 list one value per block). Read W_1 row by row: the first block row is $p = Af - Kv$, the second is the same with -1 in the bias, so $\text{relu}(p) - \text{relu}(p-1) = \text{step}(p)$; the last three block rows just copy the state. Read W_2: the new `visited` is `visited` plus $\text{relu}(p) - \text{relu}(p-1)$; the new `frontier` is just $\text{relu}(p) - \text{relu}(p-1)$; the new `dist` is `dist` $-$ `visited` $+1$.



One BFS layer. "new" is $\mathrm{relu}(p) - \mathrm{relu}(p-1) = \mathrm{step}(p)$.

To see actual neurons, here is **one node** of one layer. Take node 3 (neighbours 1, 2, 4) and compute its entry of new:



$p_3 = f_1 + f_2 + f_4 - 3v_3$; the lower hidden neuron has bias -1. The whole layer is 8 of these side by side, plus copy neurons.

The same (W_1, b_1, W_2, b_2) is applied 7 times. Cost: **40 hidden neurons per layer** ($2V$ for the step, $3V$ for the copies), 7 layers.

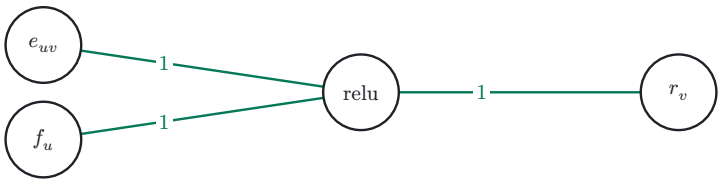
Part 2: the graph is an input too

In part 1 the graph is a constant, so Af is a **constant matrix times a runtime vector**: free. Now let the edges be runtime bits $e_k \in \{0, 1\}$ (one per possible edge; 5 nodes have 10, so $2^{10} = 1024$ graphs). The product is no longer constant × runtime but runtime × runtime: "$e_{uv} \cdot f_u$". Product of two 0/1 numbers is an AND, and AND is one ReLU:

$$\mathrm{AND}(e_{uv}, f_u) = \mathrm{relu}(e_{uv} + f_u - 1)$$

For every edge and each of its two directions, one such neuron. $\mathrm{reached}_v$ is then the sum of the AND neurons that point into v:

$$\mathrm{reached}_v = \sum_u \mathrm{relu}(e_{uv} + f_u - 1), \quad \mathrm{new}_v = \mathrm{step}(\mathrm{reached}_v - K \cdot \mathrm{visited}_v)$$



One AND neuron (bias -1): fires only when the edge is present **and** u is on the frontier.

The wave now needs **two** hidden layers instead of one: first the AND neurons (and reached as their sum), then the step neurons that need reached. $K = V - 1 = 4$ is now the bound ($\mathrm{reached}_v$ counts at most $V - 1$ incoming edges). Cost for 5 nodes: block 1 has 2×10 AND neurons plus 25 copies = 45; block 2 has $2V = 10$ step neurons plus 25 copies = 35; so 80 hidden neurons per wave, 4 waves, 8 hidden layers. The check runs all $2^{10} = 1024$ graphs × 5 start nodes = 5120 cases.

B · branchless arithmetic

```
for k, (u, v) in enumerate(PAIRS):
    # both runtime: AND = relu(p + q - 1)
    reached[v] += relu(
        e[k] + frontier[u] - 1)
    reached[u] += relu(
        e[k] + frontier[v] - 1)
```

C · network (matrices + relu)

```
# block 1: one row per edge direction
W1a[row, k] = 1           # e_k
W1a[row, FRO + src] = 1   # frontier_src
b1a[row] = -1
W2a[NX + dst, row] = 1    # reached_dst
```

What to notice

- **The graph structure is just weights.** A fixed graph is a constant matrix A in W_1; to change the graph you change numbers, not code. A runtime graph costs one AND neuron per edge direction instead.
- **No queue, no `while`.** The frontier is a set; the number of waves is fixed by $V - 1$.
- **K is a bound you derive,** not a magic number: max degree (fixed graph) or $V - 1$ (runtime graph).
- **Everything is ≥ 0,** which is what lets $\text{relu}(x) = x$ serve as a "wire". A negative state variable (like a dist of -1) would be clipped, so the -1 is only produced at the very end by an affine read-out.
- **Cost grows with the graph:** $O(V)$ neurons per layer for a fixed graph, $O(V^2)$ AND neurons for a runtime graph.

Try it

Exercise 1 (answer at the end of the chapter). With $K = 1$ instead of $K = 3$, find a start node and a wave of the example graph where a visited node is "discovered" again. Then find the start node that really needs $K = 3$.

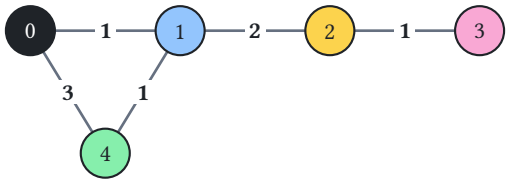
15 Shortest paths: relaxation instead of a heap

`basic_python/04_graph_search/15_shortest_path.py`

The problem

Same question as BFS, but every edge has a **weight**: a small positive integer (1 to 3 here). Find the cheapest total weight from the start to every node, and -1 if unreachable. BFS is no longer enough: a path with more edges can be cheaper than one with fewer.

Inputs: one weight per possible undirected edge of 5 nodes (10 numbers in $\{0, 1, 2, 3\}$, where 0 means **no edge**) and the start node as a one-hot vector. The graph is an **input** from the very start, not baked into the weights.



The "trap" graph used below. Colour = final distance from node 0 (dark 0, blue 1, green 2, amber 3, pink 4). The direct edge $0 - 4$ costs 3, but going through node 1 costs $1 + 1 = 2$.

Two textbook algorithms

A · control flow

Dijkstra (this is `control_flow`)

```
def dijkstra(weights, start):
    dist = [-1] * V
    heap = [(0, start)]
    while heap:
        d, u = heappop(heap)
        if dist[u] != -1:
            continue
        dist[u] = d
        for v in range(V):
            w = weight(u, v)
            if w > 0 and dist[v] == -1:
                heappush(heap, (d + w, v))
    return dist
```

A · control flow

Bellman-Ford

```
def bellman_ford(weights, start):
    d = [inf] * V
    d[start] = 0
    for _ in range(V - 1):
        for u in range(V):
            for v in range(V):
                w = weight(u, v)
                if w > 0 and d[u] + w < d[v]:
                    d[v] = d[u] + w
    return [x if x != inf else -1
            for x in d]
```

Key idea

Dijkstra's heap **pops one node per step** and the pop order depends on everything pushed so far: inherently sequential, and "pop the smallest" is a data-dependent branch. Bellman-Ford has no queue. Every round it relaxes **every edge**, with the same arithmetic on all of them, so a round is one layer of data flow. It does more work ($V - 1$ rounds of E edges instead of $E \log V$), but it is the only one of the two whose shape is a fixed repetition of the same formula.

Why $V - 1$ rounds? After round r, d_v is at most the cost of the cheapest path that uses at most r edges (induction: a path of r edges is a path of $r - 1$ edges plus one relaxed edge). A shortest path never repeats a node, so it has at most $V - 1$ edges.

The relaxation layer

$$d'_v = \min\Big(d_v, \min_{u \neq v}(d_u + c_{uv})\Big)$$

where c_{uv} is the weight of the edge, or INF when there is none. Every round uses the **old** d for all nodes at once (Bellman-Ford with all edges relaxed in parallel); A uses the in-place variant, but the final answers are identical, and the check asserts that Dijkstra, Bellman-Ford, B and C all agree.

Weights are inputs, and that costs nothing. $d_u + c_{uv}$ is a plain addition. Unlike BFS part 2 there is no AND and no `select`: two runtime values meet only in a sum, and sums are affine. The only non-linearity in the whole algorithm is `min`.

No edge = INF. The input weight 0 means "no edge". Turn it into a cost that can never win a `min`:

$$c = w + \text{INF} \cdot (1 - \text{step}(w))$$

For $w \in \{1, 2, 3\}$, $\text{step}(w) = 1$ and $c = w$. For $w = 0$, $\text{step}(0) = 0$ and $c = \text{INF}$. The start node gets $d = 0$, every other node gets INF: $d = \text{INF} \cdot (1 - s)$ for the one-hot s.

How big must INF be? INF is "a finite stand-in for infinity". Two requirements:

Every real value is below INF. A real distance estimate is the length of an actual walk of at most V edges of weight ≤ 3, so at most $V \cdot 3 = 15$. Taking $\text{INF} = V \cdot \text{WMAX} + 1 = 16$ keeps every real value strictly below INF, so "$d \geq \text{INF}$" can mean "unreachable".	**Fake values never win.** A candidate built from a missing edge is $d_u + \text{INF} \geq \text{INF}$. A candidate through an unreached node is $\text{INF} + c \geq \text{INF} + 1$. Either way it is not smaller than $d_v \leq \text{INF}$, so the `min` ignores it. An unreachable node stays at exactly INF.

Too small an INF breaks things silently (see "Try it"). Too big is harmless apart from larger numbers (the biggest value that ever appears is $2 \cdot \text{INF} = 32$).

Final read-out. Unreachable means $d = \text{INF}$, so $\text{reach} = \text{step}(\text{INF} - d)$ is 1 for real distances and 0 for INF, and

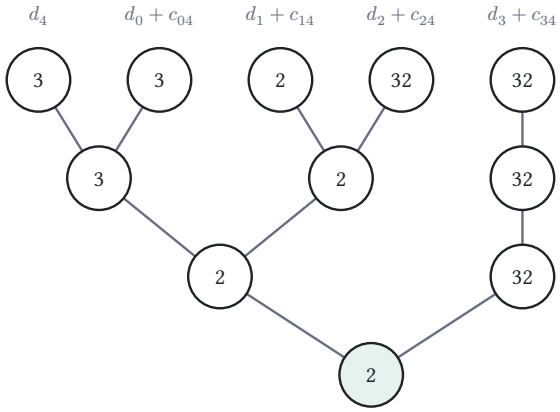
$$\text{out} = d - (\text{INF} + 1)(1 - \text{reach})$$

leaves real distances alone and maps INF to -1 (again an affine read-out, no negative number ever sits in the state).

`min` of many things: a tournament

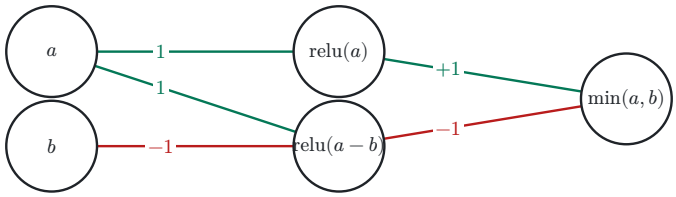
The toolbox gives `min` of two: $\min(a, b) = a - \text{relu}(a - b)$. Node v has $V = 5$ candidates (d_v and four $d_u + c_{uv}$). Pair neighbours, take the min of each pair, then pair the winners: $5 \to 3 \to 2 \to 1$, so **3 hidden layers per round** ($\lceil \log_2 5 \rceil = 3$). An odd one out just moves up a level unchanged. In stage B this is `min_tree`:

```
def min_tree(xs):
    xs = list(xs)
    while len(xs) > 1:      # the length is fixed: a fixed loop
        paired = [xs[i] - relu(xs[i] - xs[i + 1]) for i in range(0, len(xs) - 1, 2)]
        xs = paired + xs[len(xs) // 2 * 2:]    # the odd one out moves up
    return xs[0]
```



Node 4 in round 2 of the trap graph (INF = 16): $d = [0, 1, 16, 16, 3]$ and $c_{04} = 3$, $c_{14} = 1$, $c_{24} = c_{34} = 16$. Candidates $3, 3, 2, 32, 32$; the tournament returns 2.

One `min(a, b)` is three neurons' worth of wiring: a copy of a, the neuron $\text{relu}(a - b)$, and an output that subtracts:



$\text{relu}(a) = a$ because every number in the state is ≥ 0.

A and B side by side

B · branchless arithmetic

```
def branchless(weights, start):
    cost = [w + INF * (1 - step(w))
            for w in weights]
    d = [INF * (1 - s) for s in start]
    for _ in range(V - 1):
        d = [min_tree(
                [d[v]] + [d[u] + cost[idx[u, v]]
                          for u in range(V)
                          if u != v])
             for v in range(V)]
    return [d[v] - (INF + 1)
              * (1 - step(INF - d[v]))
            for v in range(V)]
```

C · network (matrices + relu)

```
LAYERS = [pre_layer()] \
       + ROUND * (V - 1) \
       + [post_layer()]

def network(weights, start):
    x = np.concatenate([weights, start])
    for layer in LAYERS:
        x = layer(x)   # W2 relu(W1 x + b1) + b2
    return x.astype(int).tolist()
```

(`if u != v` in B compares two **indices**, which are constants fixed by the graph's shape, not runtime data.)

Worked trace: the trap graph from node 0

Cost vector (c in the order of the pairs $01, 02, 03, 04, 12, 13, 14, 23, 24, 34$), from weights $[1, 0, 0, 3, 2, 0, 1, 1, 0, 0]$:

$$c = [1, \text{INF}, \text{INF}, 3, 2, \text{INF}, 1, 1, \text{INF}, \text{INF}] \quad (\text{INF} = 16)$$

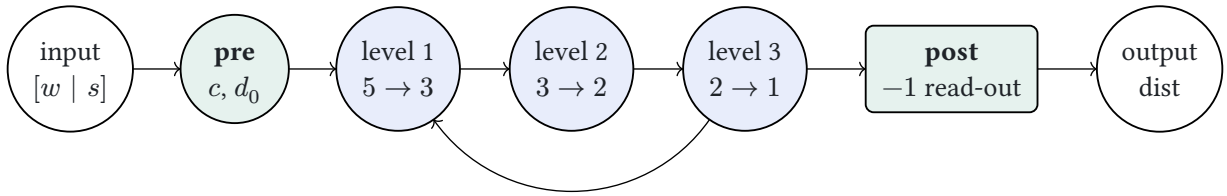
round	d_0	d_1	d_2	d_3	d_4	what happened
0	0	16	16	16	16	start: INF everywhere except the start
1	0	1	16	16	3	neighbours of 0: $d_1 = 0 + 1$, $d_4 = 0 + 3$
2	0	1	3	16	2	$d_2 = 1 + 2$; $d_4 = \min(3, d_1 + 1) = 2$: the trap is **fixed**
3	0	1	3	4	2	$d_3 = d_2 + 1$
4	0	1	3	4	2	nothing changes (spare round)

Dijkstra settles the same nodes in cost order 0, 1, 4, 2, 3 using the heap: pop $(0, 0)$; push $(1, 1)$ and $(3, 4)$; pop $(1, 1)$; push $(2, 4)$ and $(3, 2)$; pop $(2, 4)$ and so on. Bellman-Ford needed no such bookkeeping: it just improved d_4 from 3 to 2 when round 2 saw a better candidate. That is the whole difference: **a better answer overwrites a worse one** instead of **deciding the order in which answers are final**.

Round 3 repeats the same formula on the same numbers, and so does round 4: when nothing improves, nothing changes. So, like BFS, **halting is free**: the extra rounds are harmless.

Stage C: the layers

Every layer is $x' = W_2\,\text{relu}(W_1 x + b_1) + b_2$, assembled by a small `Layer` class that fills in constant matrices at import time (no computation on runtime values). Copying a number through a layer uses $\text{relu}(x) = x$, valid because every state number is ≥ 0.



repeat the 3 levels $V - 1 = 4$ times

layer	hidden neurons and what they compute	count
pre	$\mathrm{relu}(w_k)$, $\mathrm{relu}(w_k - 1)$ for each of the 10 weights (gives $c_k = (1 - \mathrm{INF})\,\mathrm{relu}(w_k) + \mathrm{INF}\,\mathrm{relu}(w_k - 1) + \mathrm{INF}$), and a copy of each start bit	25
level 1	for each node: 2 pairs × ($\mathrm{relu}(a)$, $\mathrm{relu}(a - b)$) + the odd candidate = 5; plus copies of the 10 costs	$5 \times 5 + 10 = 35$
level 2	for each node: 1 pair (2 neurons) + the odd one = 3; plus 10 copies	$5 \times 3 + 10 = 25$
level 3	for each node: 1 pair = 2 neurons; plus 10 copies	$5 \times 2 + 10 = 20$
post	$\mathrm{relu}(\mathrm{INF} - d)$, $\mathrm{relu}(\mathrm{INF} - d - 1)$, and a copy of d, per node	15

The first level's candidates are **affine functions of the state** $[d \mid c]$: candidate $d_u + c_{uv}$ is the row vector with a 1 at d_u and a 1 at c_{uv}. So adding the weight and taking the first `min` happen in the same layer: the matrix W_1 of level 1 is built from rows like "$(d_u + c_{uv}) - d_v$".

Total: $25 + 4 \times (35 + 25 + 20) + 15 = 360$ hidden neurons in 14 layers. The test compares Dijkstra, Bellman-Ford, B and C on 25 hand-picked (weights, start) cases (no edges, all weight 1, a path of weight-3 edges whose longest distance is 12, the trap, all edges weight 3) and on 2000 random weight vectors of three densities.

What to notice

- **Pick the algorithm whose shape is data flow.** Dijkstra and Bellman-Ford compute the same thing; only one of them is a repeated formula on all nodes. When you design your own algorithm, look for a "relax everything" formulation first.
- **`min` is the only non-linearity.** Sums are free; so are the weights, even though they are runtime inputs.
- **INF is a bound you derive.** $\mathrm{INF} > V \cdot \mathrm{WMAX}$ is enough; the derivation is two inequalities.
- **Depth vs width of `min`.** A tournament costs $\log_2 n$ layers and about $2n$ neurons per node. The "compare every pair" approach (used by `argmin` in ch. 3 and in A* below) costs 2 layers but about n^2 neurons. You choose a point on that curve.
- **Negative weights** would still work for Bellman-Ford in control flow, but this network keeps every state value ≥ 0 so that copies are free; you would have to offset the distances to keep that true.

Try it

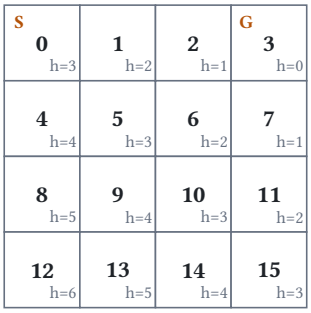
Exercise 2 (answer at the end of the chapter). The weights grow to $\mathrm{WMAX} = 5$. What are INF and the number of rounds? Then: what would go wrong with $\mathrm{INF} = 10$ if the weights form a path $0 - 1 - 2 - 3 - 4$ with all weights 3?

16 A*: the same sequential algorithm, one step per layer

`basic_python/04_graph_search/16_astar.py`

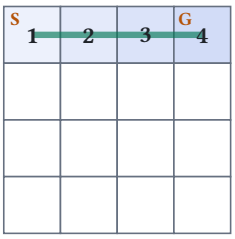
The problem

A 4×4 grid. Nodes are numbered row by row, 0 to 15. The **start** is node 0 (top-left), the **goal** is node 3 (top-right). Moves go up, down, left, right at cost 1. Each of the other cells is a runtime input bit, 1 = wall. (A wall bit on the start is ignored, since the start is expanded first; a wall on the goal makes it unreachable.) A* finds a cheapest path using a heuristic h_v = Manhattan distance to the goal, which is a **constant vector** because the goal is fixed:

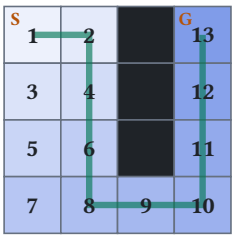


Node numbers and h_v. $h_v = |\text{row}_v - 0| + |\text{col}_v - 3|$, so h is 0 at the goal and 6 at the bottom-left corner. g = cost of the best route found so far, $f = g + h$.

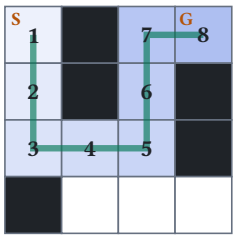
The result is a triple, identical in all three stages: **(cost or −1, the order in which nodes were expanded, the path from goal to start)**. Lists are padded with −1 up to 16 entries so that A, B and C can be compared with ==.



no walls: A* walks straight to the goal



a wall with a gap at the bottom: A* first runs into the wall



a small maze

Numbers = order in which A* expanded each cell (1 = the start). Green line = the returned path. Blank cells were never expanded.

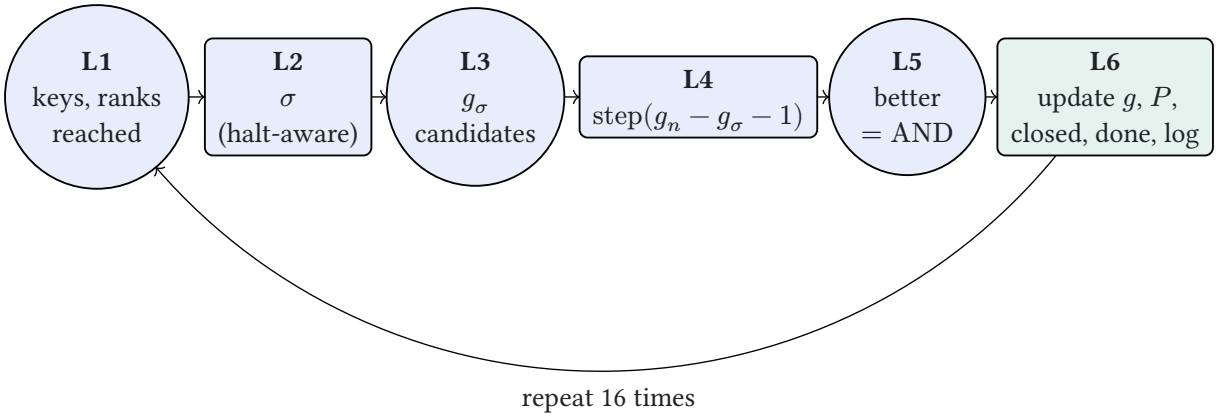
What stays sequential, and what does not

A* is **inherently sequential**: each expansion depends on the previous one (which node has the best f right now?). We do not fight that. The control-flow algorithm does at most one expansion per loop trip and there are at most 16 nodes, so the data-flow version is **16 identical steps**, each step one expansion. Inside one step everything is parallel over nodes: all keys at once, all neighbours at once.

What has to be replaced:

control flow	data flow
the heap: "pop the entry with the smallest `(f, node)`"	keys for all nodes, an `argmin` one-hot (σ)
`closed` set, `g` dict, `parent` dict	vectors: closed (0/1), g (integers), P: one one-hot row per node
`if u == GOAL: break`, `while heap:` ends	a `done` flag that freezes every later step
`order.append(u)`	a **log**: the one-hot chosen at step t is row t of a 16×16 matrix
`for n in NEIGH[u]` with ifs	a constant adjacency matrix times σ, then 0/1 masks
stale duplicates in the heap, `if u in closed: continue`	nothing: there are no duplicates, one key per node

One A* step in six layers



Stage B, line by line (C is the same arithmetic as matrices). Notation: $N = 16$ nodes, v, u, n are node indices.

1. Key and rank (argmin).

$$k_v = N \cdot (g_v + h_v + \text{BIG} \cdot \text{closed}_v) + v, \quad \text{rank}_v = \sum_{u \neq v} \text{step}(k_v - k_u)$$

BIG pushes closed nodes (and unreached ones, whose $g = \text{BIG}$) far away. This is *affine*: closed_v is a 0/1 number multiplied by the **constant** BIG, so no `select` is needed. Adding v to the key makes all keys **different**, so ties are gone: the lowest index wins automatically, exactly like the heap's `(f, node)` tuples. With unique keys, $\text{step}(k_v - k_u)$ is exactly "u has a smaller key than v", and rank_v counts the nodes that beat v. Rank 0 means "I am the minimum". Cost: two neurons per ordered pair, $2N(N-1) = 480$.

Bounds. A path never repeats a node, so $g \leq 15$; $h \leq 6$; so every real $f = g + h \leq 21$. We need an open node to always beat closed and unreached ones: $\text{BIG} = 15 + 6 + 1 = 22$. Then k stays below $16 \cdot 43 + 15 = 703$ and all differences are small integers, so `step` is exact.

node is	$g + h + \text{BIG} \cdot \text{closed}$	compared with the open nodes
open	≤ 21	it is the competition
closed	≥ 22	always loses
unreached ($g = \text{BIG}$)	≥ 22	always loses

2. Select, with the halt flag.

$$\text{open}_v = \text{step}(\text{BIG} - g_v) - \text{closed}_v, \quad \sigma_v = \text{relu}(\text{open}_v - \text{done} - \text{rank}_v)$$

$\text{step}(\text{BIG} - g_v)$ is 1 for nodes that have been reached ($g < \text{BIG}$). Then $\sigma_v = 1$ exactly when **all three** hold: rank 0, open, and `done` is 0. Why one ReLU is enough: $\text{relu}(\text{open} - \text{done} - \text{rank})$. If the rank is 0 this is $\text{relu}(\text{open} - \text{done})$, which is 1 only for open = 1 and done = 0 (an AND). If the rank is 1 or more, then $\text{open} - \text{done} - \text{rank} \leq 1 - 0 - 1 = 0$, so the output is 0.

Why this one line is the whole halting mechanism:

- **Nothing is open** (the goal is walled in): the argmin picks some closed or unreached node, but its open is 0, so $\sigma = 0$ everywhere.
- **Already done**: done = 1 turns every σ_v off.
- Whenever $\sigma = 0$, every later formula (closing, relaxing, logging) is applied with a zero vector: **the state is frozen.** That is the same "halted flag freezes the state" idea as chapter 2, but here it costs no extra layers.

3. Update `done`.

$$\text{done} \leftarrow 1 - \sum_v \sigma_v + \sigma_{\text{goal}}$$

situation	$\sum\sigma$	σ_{goal}	new done
already done	0	0	$1-0+0=1$
nothing open	0	0	1
expanded a non-goal node	1	0	$1-1+0=0$
expanded the goal	1	1	$1-1+1=1$

4. Read g of the selected node.

$$g_\sigma = \sum_v \text{relu}(g_v - \text{BIG}\cdot(1-\sigma_v))$$

This is chapter 3's masked read (`xs[i]` with a one-hot `i`): the neuron for node v passes g_v only when $\sigma_v = 1$. Because $g_v \geq 0$ one ReLU suffices, with $M = \text{BIG}$ (the largest g_v).

5. Neighbours of the selected node, and which may be relaxed.

$$\text{near} = A_{\text{grid}}\,\sigma, \quad \text{cand}_n = \text{relu}(\text{near}_n - \text{wall}_n - \text{closed}_n)$$

A_{grid} is the constant 0/1 adjacency matrix of the grid (the same trick as BFS), with the goal's column zeroed so the goal never expands. $\text{cand}_n = 1$ exactly when n touches the selected node, is not a wall and is not closed (three 0/1 conditions: subtract the "bad" ones from the "good" one and clip).

6. Did we find a better route?

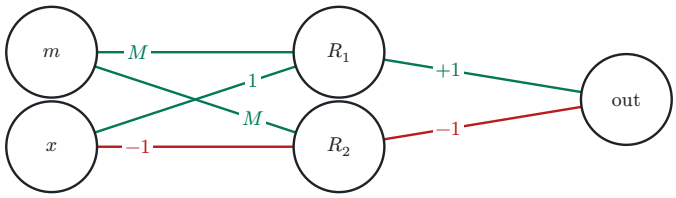
$$\text{better}_n = \text{relu}(\text{cand}_n + \text{step}(g_n - g_\sigma - 1) - 1)$$

$g_\sigma + 1 < g_n$ is $g_n - g_\sigma - 1 \geq 1$ (integers), and $\text{relu}(p+q-1)$ is the AND. (Unreached nodes have $g_n = \text{BIG} \geq 22 > 16 \geq g_\sigma + 1$.)

7. Overwrite g and the parent with big-M selects.

$$g'_n = g_n + \text{select}_M(b_n, g_\sigma + 1 - g_n), \quad P'_{nu} = P_{nu} + \text{select}_M(b_n, \sigma_u - P_{nu})$$

with $b_n = \text{better}_n$, $M = \text{BIG}$ for g and $M = 1$ for P, and $\text{select}_M(m, x) = \text{relu}(x - M(1-m)) - \text{relu}(-x - M(1-m))$ from the toolbox. Here the 0/1 mask b_n multiplies a **runtime** difference, which is exactly what `select` is for. Bounds: $1 \leq g_\sigma + 1 \leq 16$ and $0 \leq g_n \leq 22$, so $|x| \leq 22 = M$ for g; and $|\sigma_u - P_{nu}| \leq 1 = M$ for the parent bits. The parent register P has one one-hot row per node: row n says which node is the parent of n (all zeros = none yet).



Big-M select: $R_1 = \text{relu}(x + Mm - M)$ and $R_2 = \text{relu}(-x + Mm - M)$. Used 16 times for g and 256 times for P per step.

8. Close, log.

$$\text{closed}' = \text{closed} + \sigma, \quad \text{log}' = [\text{rows 2 to 16 of log}, \sigma]$$

The log is a **shift register**: drop the oldest row, append σ as the newest. After 16 steps, row t is the node expanded at step t (an all-zero row if the search had already stopped). Shifting keeps all 16 steps identical, no layer needs to know its step number.

Path reconstruction

After the search, follow the parent pointers from the goal. The pointer chase is itself a fixed loop (a path has at most 16 nodes):

$$\text{cur}_0 = \text{found} \cdot e_{\text{goal}}, \quad \text{cur}_{t+1,u} = \sum_v \text{AND}(P_{vu}, \text{cur}_{t,v}) = \sum_v \text{relu}(P_{vu} + \text{cur}_{t,v} - 1)$$

where $\text{found} = \text{closed}_{\text{goal}}$ (0/1) times the **constant** one-hot e_{goal}. cur is a one-hot "where am I"; the sum over v is "parent of the node I am on", an array read with the one-hot as address, done with one AND neuron per (v, u) pair, 256 per step. The start has no parent (its row of P is zero), so cur becomes the zero vector and stays there: padding for free. The 16 cur vectors are logged with the same shift register. The **cost** is read as $g_{\text{goal}} - (\text{BIG} + 1)(1 - \text{found})$, which is -1 if the goal was never expanded (then $g_{\text{goal}} = \text{BIG}$). In the network it is stored as cost $+1$ so that it stays ≥ 0.

Worked trace: the wall with a gap

Grid: walls at 2, 6, 10 (the third column, top three rows). Start 0, goal 3 is right behind the wall. The first steps of A* (this is what each of the 16 layers computes; open nodes are written `node: g + h = f`):

step	open set before the step	selected	relaxed (new g)
1	`0: 0+3=3`	0	`1: 1+2=3, 4: 1+4=5`
2	`1: 3, 4: 5`	1	`5: 2+3=5`
3	`4: 5, 5: 5`	4 (tie)	`8: 2+5=7`
4	`5: 5, 8: 7`	5	`9: 3+4=7`
5	`8: 7, 9: 7`	8 (tie)	`12: 3+6=9`
6	`9: 7, 12: 9`	9	`13: 4+5=9`
7	`12: 9, 13: 9`	12 (tie)	none: 13 already has $g = 4$
8	`13: 9`	13	`14: 5+4=9`
9 to 12	one open node each time, $f = 9$	14, 15, 11, 7	15 (g 6), 11 (g 7), 7 (g 8), 3 (g 9)
13	`3: 9`	3 = goal	no relax; `done` becomes 1
14 to 16	(frozen)	none	nothing: $\sigma = 0$

A* follows the heuristic straight into the wall: cells 0, 1, 4, 5, 8, 9 are cheap (f = 3 to 7). Then the $f = 9$ plateau opens: every remaining step has the same f, and ties go to the lowest index (12 before 13). It then walks the long way round the bottom of the wall and up. The returned cost is 9; the path is $3, 7, 11, 15, 14, 13, 9, 5, 1, 0$ goal to start.

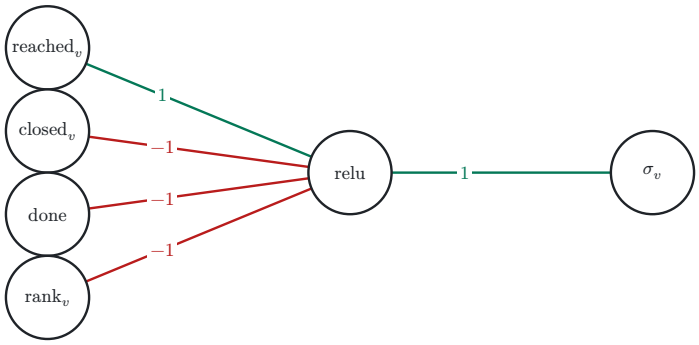
Zoom in on step 3, the first tie. Open: node 4 ($g = 1, h = 4$) and node 5 ($g = 2$, $h = 3$), both $f = 5$. The keys are $k_4 = 16 \cdot 5 + 4 = 84$ and $k_5 = 16 \cdot 5 + 5 = 85$; everything else is above 350 (closed nodes carry $+22$ inside the bracket, unreached ones have $g = 22$). So $\text{rank}_4 = 0$ and $\text{rank}_5 = 1$. Then $\sigma_4 = \text{relu}(1 - 0 - 0 - 0) = 1$ and $\sigma_5 = \text{relu}(1 - 0 - 0 - 1) = 0$: node 4 is selected, same as the heap's `(5, 4) < (5, 5)`.

Stage C: the network

The whole state is one vector of 658 numbers, all ≥ 0 (so copies are free, as before). **Persistent** registers come first, **scratch** registers (reset to 0 every step) after them:

register	size	role
g	16	best cost so far; BIG = "not reached"
closed	16	0/1
done	1	0/1 halt flag
wall	16	the runtime input, carried along
P	256	parent one-hot per node
log	256	expansion order, shift register
scratch: rank, reached, σ, g_σ, cand, stp, better	97	intermediate values between the six layers

Each layer is $x' = W_2 \,\text{relu}(W_1 x + b_1) + b_2$ and is built at import time, one hidden neuron per formula above, by the same small `Layer` class as in example 15. For instance $\sigma_v = \text{relu}(\text{reached}_v - \text{closed}_v - \text{done} - \text{rank}_v)$ is one row of W_1 in layer L2:




```
# layer L2, as written in the file: sel[v] = relu(reached - closed - done - rank)
for v in range(N):
    row = row_of(REACHED=(v, 1), CLOSED=(v, -1), DONE=(0, -1), RANK=(v, -1))
    L2.output(at("SEL", v), [(L2.unit(row), 1)])
```

Neuron counts (hidden neurons per layer): L1 1073 (480 pairwise + 32 reached + 561 copies), L2 577, L3 609, L4 626, L5 594, L6 1104 (with the 16 g selects, 256 P selects, and copies). One step is 4583 neurons. Most of them are **copies** of the persistent registers (P and the log are 512 numbers carried through every layer); a residual connection would remove them, but then the layer would no longer be a plain "matrix, ReLU, matrix". With 16 steps and the 16-layer path walk (1025 neurons per layer): about $16 \times 4583 + 16 \times 1025 \approx 90000$ neurons in $16 \times 6 + 16 = 112$ layers.

A · control flow

```
f, u = heappop(heap)
if u in closed:
    continue
closed.add(u)
order.append(u)
if u == GOAL:
    break
```

B · branchless arithmetic

```
key = [N * (g[v] + H[v]
           + BIG * closed[v]) + v
       for v in range(N)]
rank = [sum(step(key[v] - key[u])
           for u in range(N) if u != v)
        for v in range(N)]
sel = [relu(reached[v] - closed[v]
           - done - rank[v])
       for v in range(N)]
```

A · control flow

```
for n in NEIGH[u]:
    if walls[n] or n in closed:
        continue
    if g[u] + 1 < g.get(n, BIG):
        g[n] = g[u] + 1
        parent[n] = u
        heappush(heap,
                 (g[n] + H[n], n))
```

B · branchless arithmetic

```
near = [sum(sel[u] for u in NEIGH[n]
           if u != GOAL) for n in ...]
cand = [relu(near[n] - walls[n]
            - closed[n]) for n in ...]
better = [relu(cand[n]
    + step(g[n] - g_sel - 1) - 1)
    for n in range(N)]
g[n] += select(better[n],
               g_sel + 1 - g[n], BIG)
P[n][u] += select(better[n],
                  sel[u] - P[n][u], 1)
```

The check runs 1006 wall layouts: six hand-picked (no walls, the wall with a gap, goal walled in, start walled in, the goal itself a wall, a small maze) and 1000 random ones with wall probability 0.1 to 0.4. For every one, A, B and C agree on cost, expansion order and path, and the cost also equals plain BFS's.

What to notice

- **A sequential algorithm becomes a fixed number of identical layers.** The price is a step count you must bound ahead of time (16 expansions at most), and the fact that the layers after the end still exist (frozen by `sel = 0`).
- **Pick-the-best is `argmin`, and the tie-break is part of the key.** $N \cdot \text{key} + \text{index}$ makes keys unique; any tie rule is just a different low-order term.

- **"Push it far away" is affine when a 0/1 bit multiplies a constant;** `select` is only needed when a 0/1 bit multiplies a runtime value.
- **Every subtraction of a bad flag is a hidden AND/NOT.** $\mathrm{relu}(\mathrm{near} - \mathrm{wall} - \mathrm{closed})$ is one neuron instead of three.
- **Derive BIG and every M from the bounds on g and h;** write them next to the code. They are the first thing to check when a test fails.
- **Consistent heuristic:** Manhattan distance with unit moves never needs a closed node to be reopened, which is why the `n in closed` filter is safe. With an inconsistent h you would remove that filter (the `better` test alone then decides).
- **Making the goal a runtime input** only changes h: read the goal's row and column with a one-hot and compute $|r_v - r_g| + |c_v - c_g|$ with $|x| = \mathrm{relu}(x) + \mathrm{relu}(-x)$, about 4 more neurons per node. Everything else stays.

Try it

Exercise 3 (answer at the end of the chapter). Change the tie-break to prefer the **highest** node index among equal f. Which places in stage A and in the key formula change? And what changes if you set $h = 0$ everywhere (and what can BIG shrink to)?

A recipe for your own search algorithm

Every algorithm in this chapter fits one template. To design BFS, Dijkstra, greedy best-first, a maze solver or anything similar by hand:

1. **Choose the state vectors.** One entry per node for every fact the algorithm remembers: `visited/closed` (0/1), `frontier/open` (0/1), a cost (`dist`, `g`), a parent (one-hot per node), a log of what happened in order. **Keep every number ≥ 0 and bounded,** store offsets if you must ("cost + 1"), and write the bounds down: they give you K, INF, BIG and every M.
2. **Choose the select rule: what does a layer process?** This is the only thing that really differs between algorithms.

algorithm	selected set	how it is built
BFS	the whole frontier	the frontier vector itself; no search needed
Bellman-Ford	every edge	nothing: relax everything, `min` decides
Dijkstra	one node: smallest g	argmin one-hot, key $= g$ (+ BIG on closed)
A*	one node: smallest f	argmin one-hot, key $= g + h$
greedy best-first	one node: smallest h	argmin one-hot, key $= h$
DFS	one node: newest open	argmax of "discovery time" (argmin of its negative)

   For a single node, build keys, make them unique with $N \cdot \text{key} + \text{index}$, and use $\text{rank}_v = \sum_{u \neq v} \text{step}(k_v - k_u)$, $\sigma_v = \text{relu}(1 - \text{rank}_v)$ (2 layers, $O(N^2)$ neurons). For a whole set, skip all that.
3. **Expand along the edges.** Fixed graph: near $= A\,\sigma$ with the adjacency matrix as weights. Graph as input: one AND neuron $\text{relu}(e_{uv} + \sigma_u - 1)$ per edge direction. Then **filter**: for each node, subtract the 0/1 flags that forbid it ($\text{relu}(\text{near} - \text{wall} - \text{closed})$), or use $\text{AND} = \text{relu}(p + q - 1)$ for two flags.
4. **Update the state with select.** A cost update is `new = old if not better else candidate`: compute $\text{better} = \text{step}(\text{old} - \text{candidate})$ and $\text{old} + \text{select}_M(\text{better}, \text{candidate} - \text{old})$ with M from the bounds. Parents are the same select on one-hot rows. Visited-style flags are just $+\sigma$ or $+$ new.
5. **Add the halt flag.** Put $-\text{done}$ inside the ReLU that builds σ, so a finished search selects nothing, and set $\text{done} \leftarrow 1 - \sum \sigma + \sigma_{\text{goal}}$. A zero selection leaves the whole state unchanged, so extra layers are harmless and you only need an **upper bound** on the number of steps. (For BFS and Bellman-Ford the process dies out on its own.)
6. **Count layers and neurons.** Steps × layers per step (2 to 6) × neurons per layer. The pairwise `argmin` is $2N(N-1)$; an adjacency matrix is N neurons per layer for a fixed graph and $O(N^2)$ AND neurons for a runtime graph; copying a register is one neuron per entry.
7. **Read out with affine maps, and test.** Decode one-hots with $\text{onehot} \cdot (1, \ldots, N) - 1$ so that "none" is -1; map "never reached" markers (INF, BIG) to -1 with an affine correction, never a comparison. Then do what every example here does: write the plain Python version first, compare against it on **every** input you can afford and on a fixed-seed random sample plus the awkward cases (no path, blocked start, ties).

Skeleton of one step, in the notation of this chapter:

```
state = initial(inputs)                          # affine in the inputs
for _ in range(MAX_STEPS):                       # fixed count
    key    = affine(state)                       # push excluded nodes to BIG
    sel    = relu(open - done - rank(key))       # select-best AND not done
    near   = ADJ @ sel                           # expand: constant matrix
    cand   = relu(near - forbidden)              # filter: 0/1 flags
    better = relu(cand + step(old - new) - 1)    # improves?
    state  = update(state, sel, better)          # big-M selects, closed += sel
    done   = 1 - sum(sel) + sel[goal]            # halt flag
```

Pitfalls checklist.

- A state value that can go negative is clipped by a "copy" neuron. Offset it, or avoid carrying it through a layer.
- `step` is exact only for integers. Keep every cost an integer.
- Ties silently change the answer. Make the tie-break part of the key and use the same one in stage A.

- `select` with a too-small M gives wrong answers only on rare inputs. Derive M from the bounds.
- Never multiply two runtime values. AND them (0/1 bits), select them (0/1 mask × value), or add them.

Answers to the exercises

1 (BFS, $K = 1$). Start node 0, wave 2. The frontier is $\{1, 2\}$, $Af = [2, 1, 1, 2, 0, \ldots]$ and visited $= [1, 1, 1, 0, \ldots]$. With $K = 1$: $p = [1, 0, 0, 2, 0, \ldots]$, so $\mathrm{step}(p) = [1, 0, 0, 1, 0, \ldots]$: node 0, already visited, is discovered again (and `visited[0]` becomes 2). With $K = 3$: $p = [-1, -2, -2, 2, \ldots]$, correct. K must be at least the largest number of frontier neighbours a visited node can have. From start node 3, wave 2: the frontier is $\{1, 2, 4\}$, all three touch node 3, so $\mathrm{reached}_3 = 3$ and $K = 3$ (the maximum degree) is really needed; $K = 2$ would rediscover node 3.

2 (shortest path). $\mathrm{INF} = V \cdot \mathrm{WMAX} + 1 = 5 \cdot 5 + 1 = 26$; the rounds stay $V - 1 = 4$ (they depend on the number of nodes, not the weights). With $\mathrm{INF} = 10$ and a path of weight-3 edges, node 4 has the true distance $4 \cdot 3 = 12 \geq 10$. Node 4 starts at $d = \mathrm{INF} = 10$, and the candidate $d_3 + 3 = 12$ is not smaller than 10, so the `min` never writes it: node 4 stays at INF and the read-out reports -1 (unreachable), although a path exists. The bound $\mathrm{INF} > V \cdot \mathrm{WMAX}$ is what keeps real values and fake values separable.

3 (A*). Prefer the highest index on ties: in stage A push $(f, -\mathrm{node})$ (both the initial entry and the relaxation push) and negate again after popping; in the key use $N \cdot \mathrm{key} + (N - 1 - v)$ instead of $+v$. The selection formulas stay the same. With $h = 0$ everywhere, the key is just g (plus the closed offset): A* becomes Dijkstra on a unit-cost grid, which expands cells in BFS layers. Then $h_{\max} = 0$, so $\mathrm{BIG} = 15 + 0 + 1 = 16$ already suffices.